



CAPACITACIÓN RPA

**MANEJO DE EXCEPCIONES: TRY CATCH, RETRY
SCOPE, GLOBAL HANDLER, THROW & RETHROW.**

Expectativas de Formación

Problema

Desconocimiento de las herramientas que se utilizan para el manejo de excepciones

Objeto

Mecanismos para el manejo de Excepciones en UiPath

Formas / Técnicas

Prácticas de las herramientas mencionadas en la presentación y sus usos

Evaluación

Preguntas específicas y ejercicios para realizar con lo aprendido

Objetivo

Identificar y diferenciar entre los diferentes tipos de excepciones; así mismo utilizar TryCatch , Throw y Rethrow en los proyectos de automatizaciones.

Contenidos

Definiciones
TryCatch
RetryScope
Global Handler
Ejercicio

ANTES DE COMENZAR

- ¿Qué es un error y qué es una excepción?

Errores

Son eventos que un programa normalmente no puede manejar. Hay diferentes tipos de errores, por ejemplo:

Syntax Errors: Donde el compilador no puede analizar/interpretar el código escrito.

User Errors: Donde se determina que el input del usuario no es aceptable por algún motivo.

Programming errors: Donde el programa no contiene errores de sintaxis pero no produce los resultados esperados.

Excepciones

Son eventos que son reconocidos por el programa, categorizados y manejados. Más específicamente, hay una rutina configurada por el desarrollador que se activa cuando se detecta una excepción. A veces, el mecanismo de manejo puede ser simplemente detener la ejecución.

Algunas de las excepciones están vinculadas a los sistemas utilizados, mientras que otras están vinculadas a la lógica del proceso.

ANTES DE COMENZAR

● System exceptions

Es un evento relacionado con la ejecución de un programa, son un tipo de interrupción para notificar al sistema de un suceso excepcional, por ejemplo, cuando se hacen operaciones con una variable null.

Nota: Todos estos tipos de excepciones que se mencionan a continuación se derivan de `System.Exception`, por lo que usar este tipo genérico detectará todos los tipos de errores.

- `NullReferenceException`
- `IndexOutOfRangeException`
- `ArgumentException`
- `SelectorNotFoundException`
- `ImageOperationException`
- `TextNotFoundException`
- `ApplicationException`

ANTES DE COMENZAR

● Business exceptions

Una business exception se refiere principalmente a la información utilizada en un proceso automatizado.

Cuando ocurren business exception, los robots detienen el proceso y se requiere la intervención humana para abordar una excepción lanzada.

Algunos de los ejemplos de business exceptions son: ciertos datos de los que depende el proyecto de automatización están incompletos, faltan, están fuera de los límites establecidos o no pasan otra validación de datos criterios.

Las excepciones de business exception no se generan automáticamente como excepciones del sistema. Deben ser definidos por un desarrollador mediante el uso de Throw Activity y manejados dentro de TryCatch.

ANTES DE COMENZAR

● Ejercicio de entendimiento

Definir si es [Application Exception](#) o [Business Exception](#)

1. Un proyecto que usa una variable sin valor = ¿?
1. Datos con códigos de producto incorrectos = ¿?
1. Un error originado en un problema técnico = ¿?
1. Una URL debe abrirse en un navegador y esa URL no responde = ¿?
1. Las excepciones definidas por el desarrollador = ¿?
1. Un error basado en que faltan ciertos datos o están incompletos = ¿?

ANTES DE COMENZAR

● Ejercicio de entendimiento

Definir si es **Application Exception** o **Business Exception**

1. Un proyecto que usa una variable sin valor = **Application Exception**
1. Datos con códigos de producto incorrectos = **Business Exception**
1. Un error originado en un problema técnico = **Application Exception**
1. Una URL debe abrirse en un navegador y esa URL no responde = **Application Exception**
1. Las excepciones definidas por el desarrollador = **Business Exception**
1. Un error basado en que faltan ciertos datos o están incompletos = **Business Exception**

Ahora que hemos entendido los tipos de excepciones comunes, aprendamos sobre las diversas técnicas de manejo de excepciones en UiPath Studio.

MANEJO DE EXCEPCIONES

- TryCatch

La actividad TryCatch detecta un tipo de excepción específico en una secuencia o actividad y muestra una notificación de error o la descarta y continúa la ejecución.

Try: Las actividades realizadas que tienen la posibilidad de arrojar un error.

The screenshot shows a 'Try Catch' configuration window. The 'Try' section contains a 'Try to Read Data' container, which in turn contains a 'Read Range' activity. The 'Read Range' activity has three input fields: a file path '"Data\Data Info.xlsx"', a sheet name '"Sheet1"', and a range '"A1:D6"'. Below the 'Try' section, the 'Catches' section is visible, showing a table with one entry: 'IOException' with the action 'Sequence'. There is a link 'Add new catch' below the table. At the bottom, the 'Finally' section is partially visible with the label 'If'.

Catches	
IOException	Sequence

[Add new catch](#)

Finally If

MANEJO DE EXCEPCIONES

- TryCatch

Catch: La actividad o conjunto de actividades a realizar cuando ocurre una excepción.

Nota: Puede agregar múltiples excepciones y actividades.

The screenshot shows the 'Try Catch' configuration window. It is divided into two main sections: 'Try' and 'Catches'. The 'Try' section contains the activity 'Try to Read Data'. The 'Catches' section is configured for an 'IOException' and contains a 'Sequence' container. Inside the 'Sequence' container, there are two activities: 'Log Message' and 'Kill Process'. The 'Log Message' activity is set to 'Warn' level with a message template 'exception.Message + " at " + exception.Sol'. The 'Kill Process' activity is set to kill the process 'excel.exe'. At the bottom of the window, there is a 'Finally' section and an 'Add new catch' button.

Try Catch

Try *Try to Read Data*

Catches

IOException

Sequence

Log Message

Log Level: Warn

Message: exception.Message + " at " + exception.Sol

Kill Process

Process: A Process type object describing the process to be c ...

ProcessName: "excel.exe" ...

Add new catch

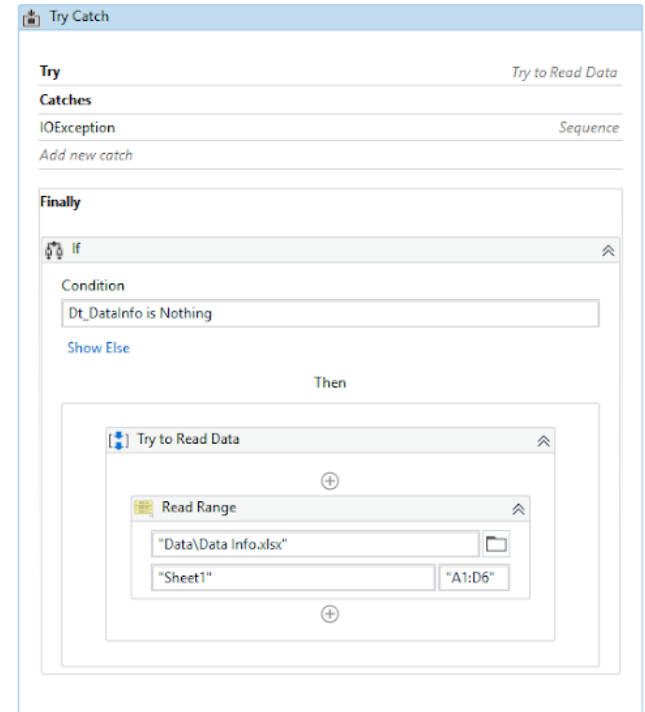
Finally *If*

MANEJO DE EXCEPCIONES

● TryCatch

Finally: La actividad o conjunto de actividades que se realizarán después de que se ejecuten los bloques Try y Catches.

Esta sección se ejecuta solo cuando no se lanzan excepciones o cuando ocurre un error y se captura en el bloque Catches (sin Rethrow).



MANEJO DE EXCEPCIONES

- TryCatch

Throw

Throw se utiliza cuando se desea generar una excepción en un momento específico.

Rethrow

Rethrow se utiliza cuando se desea realizar ciertas actividades antes de que se produzca la excepción.

Veamos un ejemplo visual de cómo funciona el
Try - Catch - Finally

¿Cuál es el límite de la cantidad de Catches que pueden usarse en una actividad de Try Catch?

MANEJO DE EXCEPCIONES

- Retry Scope

La actividad Retry Scope vuelve a intentar las actividades contenidas siempre que no se cumpla la condición o se produzca un error.

Esta actividad se usa para detectar y manejar un error, por lo que es similar a TryCatch. La diferencia es que esta actividad simplemente vuelve a intentar la ejecución en lugar de proporcionar un mecanismo de manejo más complejo.

MANEJO DE EXCEPCIONES

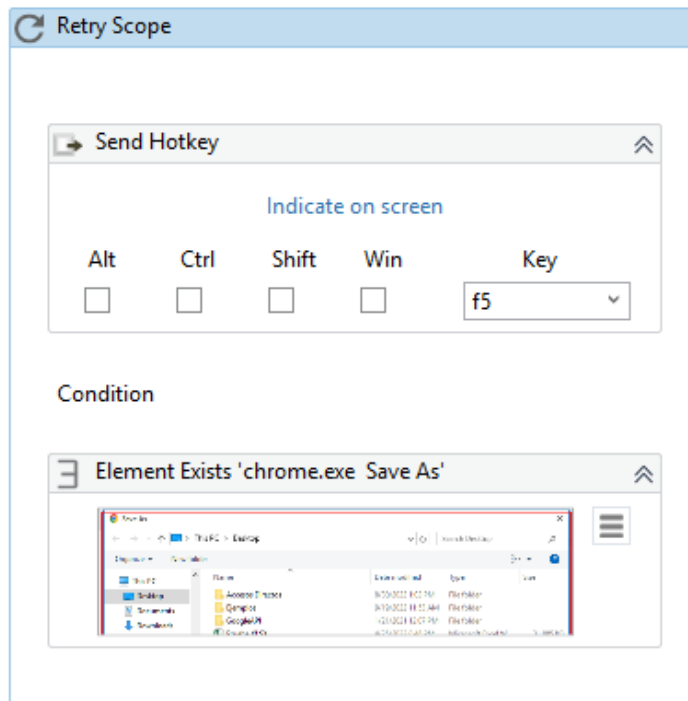
● Retry Scope

Bloques

Action: Contiene las actividades que queremos reintentar.

Condition: Condición de terminación. La actividad Retry Scope vuelve a intentar las actividades en la sección Action siempre que no se cumpla esta condición de terminación.

Nota: Solo puede tener 0 o 1 condición y debe ser boolean.



MANEJO DE EXCEPCIONES

● Retry Scope

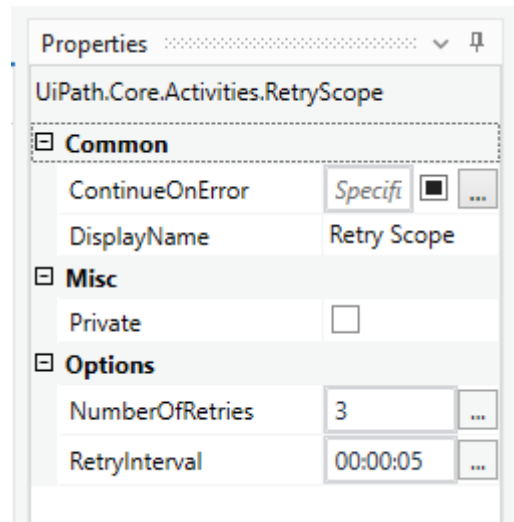
Opciones

NumberOfRetries: El número de veces que se debe reintentar la secuencia.

Default: 3 reintentos.

RetryInterval: Especifica la cantidad de tiempo (en segundos) entre cada reintento.

Default: 5 segundos.



Veamos un ejemplo visual de cómo funciona el
Retry Scope

¿En cuál de los siguientes casos el Retry Scope vuelve a intentar las actividades contenidas?

- ☐ Siempre que se arroje un error
- ☐ Siempre que se cumpla la condición
- ☐ Siempre que se cumpla la condición y se arroje un error
- ☐ Mientras no se cumpla la condición

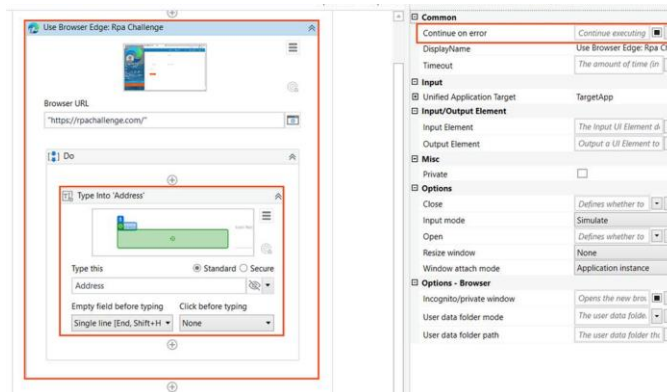
MANEJO DE EXCEPCIONES

● Continue On Error

Es una propiedad que especifica si la ejecución de la actividad debe continuar incluso cuando la actividad arroja un error.

Tener en cuenta

Si se establece en True en una actividad que contiene scope (Ej: Attach Window o Attach Browser), entonces todos los errores arrojados por las actividades dentro del DO correspondiente al scope también serán ignorados.



Continue on error: Este campo solo admite valores booleanos. Si el valor se establece en True, la ejecución del proyecto continúa independientemente de cualquier error.

Default: False.

¿Qué sucede si la propiedad ContinueOnError se establece en True?

- ☐ La ejecución del proyecto continúa independientemente de cualquier error.
- ☐ La ejecución del proyecto se detiene en función del error.
- ☐ Se vuelve a intentar la ejecución
- ☐ Se lanza un error

MANEJO DE EXCEPCIONES

- Global Handler

El Global Exception Handler es un tipo de flujo de trabajo diseñado para determinar el comportamiento del proceso cuando se encuentra con una excepción inesperada. Esta es la razón por la que solo se puede configurar un Global Exception Handler por proyecto de automatización.

En su configuración predeterminada captura las excepciones lanzadas por cualquier actividad en el proceso en tiempo de ejecución y ejecuta una respuesta estándar: ignorar, reintentar, cancelar o continuar, según lo predefinido en el momento del diseño. Para escenarios atendidos, se puede configurar para que el usuario seleccione la acción.

MANEJO DE EXCEPCIONES

● Global Handler

El controlador de excepciones global tiene 2 argumentos predefinidos, que no deben eliminarse:

errorInfo: contiene la información sobre el error que se produjo y el flujo de trabajo que falló.

result: se utiliza para determinar el siguiente comportamiento del proceso cuando encuentra el error.

Name	Direction	Argument type	Default value
errorInfo	In	ExceptionHandlerArgs	<i>Enter a VB expression</i>
result	Out	ErrorAction	<i>Default value not supported</i>
<i>Create Argument</i>			

MANEJO DE EXCEPCIONES

● Global Handler

El Global Exception Handler contiene las actividades predefinidas a continuación. Si es necesario, también podemos agregar otras actividades en nuestro Handler.

Solo las excepciones no detectadas llegarán al Exception Handler. Si se produce una excepción dentro de una actividad TryCatch y se captura y maneja con éxito dentro del bloque Catches (y no se devuelve con Retrhow), no llegará al controlador global de excepciones.

The screenshot shows the 'Global Exception Handler' configuration window. It contains two main sections: 'Log Error' and 'Choose Next Behaviour'.

Log Error: This section is used to log the error. It includes a 'Log Level' dropdown menu set to 'Error' and a 'Message' text box containing the expression `errorInfo.Exception.ToString`.

Choose Next Behaviour: This section allows the user to choose the action to be taken after the Global Exception Handler steps. It includes a list of actions: 'Continue' (The exception is re-thrown), 'Ignore' (The exception is ignored, and the execution continues from the next activity), 'Retry' (The activity which threw the exception is retried), and 'Abort' (The execution stops after running the current handler). Below this list is a 'Condition' text box containing the expression `errorInfo.RetryCount < 3`. The 'Then' and 'Else' branches are defined below the condition:

- Then:** An 'Assign' activity where the 'result' property is set to `ErrorAction.Retry`.
- Else:** An 'Assign' activity where the 'result' property is set to `ErrorAction.Abort`.

Veamos un ejemplo visual de cómo funciona el
Global Handler

¿Cuáles son las direcciones de los argumentos del Global Handler?

- ☐ errorInfo tiene dirección IN y result tiene dirección IN
- ☐ errorInfo tiene dirección de OUT y el resultado tiene dirección de OUT
- ☐ errorInfo tiene dirección IN y result tiene dirección OUT
- ☐ errorInfo tiene dirección de OUT y el resultado tiene dirección de IN

Ejercicio final para pasar el curso.

1. Cree un flujo de trabajo que tome el archivo de Excel proporcionado como entrada.
2. El flujo de trabajo agrega la diferencia entre el valor de Cash In y el valor de Cash Out para cada fila en la columna Difference.
3. Si el cálculo de la diferencia entre los dos es correcto, establezca el Status (cuarta columna) en Correcto.
4. Si no es así, establezca el Status en Cash In incorrecto o Cash Out incorrecto.
5. Utilice los bloques Try Catch para detectar excepciones al intentar convertir valores de Cash In y Cash Out.

	A	B	C	D	E
1	Month	Cash In	Cash out	Difference	Status
2	may-15	15	241	-226	Success
3	jun-15	77	90	-13	Success
4	jul-15	64	44	20	Success
5	ago-15	13K	130	N/A	Cash In wrong;
6	sep-15	144	78	66	Success
7	oct-15	160	150K	N/A	Cash Out wrong;
8	nov-15	140	13	N/A	Cash In wrong;
9	dic-15	33	49	-16	Success
10	ene-16	134	169	-35	Success
11	feb-16	37K	5K	N/A	Cash In wrong; Cash Out wrong;



